

# CostAdvisor — Project Handover Document

---

## Document Control

---

| Field             | Value                                                                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document title    | CostAdvisor — Project Handover Document                                                                                                                                                                             |
| Product / project | CostAdvisor (a StaminaChem product)                                                                                                                                                                                 |
| Document owner    | Engineering                                                                                                                                                                                                         |
| Version           | 2.1                                                                                                                                                                                                                 |
| Date              | 2026-10-02                                                                                                                                                                                                          |
| Status            | <b>Final — ready for handover.</b> Sign-Off (§15) and Roles & Responsibilities (§13) are intentionally left [TBD] for the outgoing and incoming teams to complete with real names before this is filed as accepted. |
| Classification    | Internal — Confidential                                                                                                                                                                                             |
| Distribution      | Engineering team; incoming maintainers only. Contains infrastructure, security and architecture detail — do not forward outside the team without review.                                                            |
| Related documents | CLAUDE.md , jvpdocs/remaining-work-plan.md , jvpdocs/security-posture.md , jvpdocs/eu-data-residency.md (see §14)                                                                                                   |

## Revision history

| Version | Date       | Summary                                                                                                                                                                                                                                                                                                                                                                                             |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 2026-09-27 | Initial handover snapshot, informal "mental model" format                                                                                                                                                                                                                                                                                                                                           |
| 2.0     | 2026-10-01 | Restructured into a formal handover template; document-control header added; feature-completeness/mock-data audit added; outstanding-items register and architectural follow-ups added                                                                                                                                                                                                              |
| 2.1     | 2026-10-02 | Final pre-handover pass: re-verified full test suite and clean build on this date (§8.1); re-confirmed dev fully pushed and clean; noted that the deploy-staleness commit count (§6.1, §11.1 item 1) was last measured on an earlier date and could not be independently re-verified this session (no access to the Railway-connected repo from here) — re-check the live count before acting on it |

---

## 1. Executive Summary

---

CostAdvisor is a B2B SaaS product that tells an industrial buyer what a commodity-linked product **should** cost today, given how the underlying raw-material indices have moved, so they can negotiate supplier price increases from evidence instead of guesswork.

**Engineering status: every feature across all three planned delivery waves is built, tested and merged into the `dev` branch.** The full backend test suite passes (see §8); the frontend builds clean; a feature-completeness audit (§10) confirms no screen in the application renders fabricated or mock data — every feature that was ever stubbed for specification purposes has since been replaced by a real, backend-wired implementation.

**The product is not yet live.** The Railway-connected production/staging deployment is frozen at an earlier merge and has not been synced to `dev` since go-live (§6). What stands between this codebase and a sellable, deployed product is a short list of non-engineering items — operational accounts, legal sign-offs, and acquiring real third-party data — enumerated in §11. None of it requires a further coding effort to close.

**Data residency note (EU):** production infrastructure currently runs in **US East (Virginia)**, not the EU. CostAdvisor does not today offer EU data residency; see §11.4 and `jvpdocs/eu-data-residency.md` before committing to this with any EU-headquartered prospect.

---

## 2. Purpose & Scope of This Document

---

This document hands over the CostAdvisor codebase and product to whoever picks it up next — a new engineer, a new team, or an external party. It covers:

- what the product does and how it is built (§4, §7)
- what is actually finished vs. outstanding, verified against the running code rather than taken on trust (§5, §10, §11)
- how to run, test and deploy it (§6, §8)
- the non-obvious rules that cost real debugging time if learned by breaking something (§9, §12)
- who needs to do what to take the product live (§11, §13)

It does **not** replace `CLAUDE.md`, which is the authoritative, continuously-updated engineering tracker (every scrum, what shipped, full acceptance-criteria detail) — this document is the condensed mental model and status snapshot; go to `CLAUDE.md` for the history of any specific feature.

---

## 3. How to Use This Document

---

**Reading order for a new engineer:** this document → `jvpdocs/local-setup.md` (get it running) → `jvpdocs/remaining-work-plan.md` (what is left, and three places an earlier plan's own assumptions turned out to be wrong about the data) → `CLAUDE.md` (only when you need the history of a specific scrum). `DESIGN.md` before touching any UI.

**Reading order for a non-engineering recipient** (e.g. someone assessing readiness to go live): §1, §5, §11, §13.

---

## 4. Product / System Overview

---

A supplier knows their real input costs; the buyer does not. So when the supplier says "oil went up 30%, we need +20%", the buyer has nothing to push back with. CostAdvisor closes that gap in four steps:

1. **Decompose** a product into cost components, each weighted by its share.
2. **Link** each component to a tracked commodity index, by region and quarter.
3. **Calculate** what the product *should* cost now, given how those indices actually moved.
4. **Compare** against what the supplier is charging. The difference is the negotiation signal.

Everything else in the codebase — the index layer, the catalog, the trust grading, the briefs — exists to make step 3 defensible enough to put in front of a supplier.

---

## 5. Current Status Summary

---

| Delivery wave                 | Status          | Notes                                                                                                                                                                                                                                                        |
|-------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Wave 1 — sellable core        | Code complete   | Remaining items are non-code: vendor DPAs, incident-response contacts, SMTP credentials, Search Console verification, field Core Web Vitals (§11.1)                                                                                                          |
| Wave 2 — catalog              | Code complete   | Remaining is data acquisition: base-price anchors for catalog combos, feed mapping for catalog commodities (§11.1)                                                                                                                                           |
| Wave 3 — intelligence & depth | Code complete   | Includes all 12 units of the "Index Data Layer v2" and the final eight features (data-quality console, real index forecasts, supplier price-list import, per-region sourcing, guided negotiation prep, web push, nested "Lego" cost models, AI cost modeler) |
| Deployment                    | <b>Not live</b> | Railway-connected repo is ~126 commits behind <code>dev</code> ; see §6                                                                                                                                                                                      |

A full breakdown of every still-open item, grouped by what kind of work (or non-work) it needs, is in §11. An audit confirming nothing on the frontend is mock/placeholder data is in §10.

---

## 6. Deployment, Environments & Access

---

### 6.1 Read this before you touch deploy

**The Railway-connected repo is not this repo, and it is stale.** It is frozen at the 2026-08-22 go-live merge ( `5b63c04` ); `dev` was **126 commits** past that as of the 2026-10-01 snapshot. Nothing from Scrum 26 onward, **none of the 12 "Index Data Layer v2" units**, and none of the eight features named in §5 is live. Do not verify anything infrastructure-related against the live domains and conclude the code is broken — check whether that code is even deployed first. Syncing it needs someone with access to the deploy repo and the Railway/Cloudflare dashboards (see §13).

**Caveat on the "126" figure:** this environment only has a remote for `origin` (the GitHub fork this repo lives in) — there is no remote configured here for the separate Railway-connected repo, so the count above could not be

independently re-measured on the 2026-10-02 handover date. Treat it as "last confirmed 2026-10-01," not as freshly verified today. Whoever has access to both repos should re-run the comparison before relying on the number.

### 6.2 Branches

- `dev` — the only branch with the whole picture: code plus `CLAUDE.md` , `AGENTS.md` , `jvpdocs/` , `sample_idea/` , `.claude/skills/` . **Do all work here.**
- `dev-push` , `main-push` , `main` — sanitised mirrors. A strip step removes the internal docs before push, because these are exposed to the Cloudflare Workers Builds pipeline (and `main` is production). If you were handed one of these, you cannot see this document — go get `dev` .
- To update a push branch: merge `dev` , resolve every modify/delete conflict by **keeping the deletion**, then re-run the strip as a safety net: `git rm -r --ignore-unmatch .claude 49.md AGENTS.md CLAUDE.md jvpdocs sample_folder sample_idea .` `main` additionally carries its own prod infra — `frontend/wrangler.jsonc` 's Worker name is `costadvisor-web` ; do not let a merge swap it to `dev` 's `-dev` variant or a push deploys to the wrong Worker.
- Explicit statement, not an inference:** as of this handover, `dev-push` / `main-push` / `main` are deliberately left at their stale pre-Scrum-26 point — merging `dev` into them (the sync in §6.1) was not performed as part of this handover and is a separate, explicit action for whoever takes over deploy access. Do not assume `main` reflects anything built after the 2026-08-22 go-live merge.
- Pushing to `main` / `main-push` triggers a real redeploy. Confirm with whoever owns those first.

### 6.3 Environments

|         | Production          | Staging                 |
|---------|---------------------|-------------------------|
| Landing | costadvisor.org     | dev.costadvisor.org     |
| App     | app.costadvisor.org | app.dev.costadvisor.org |
| API     | api.costadvisor.org | api-dev.costadvisor.org |
| Branch  | <code>main</code>   | <code>dev</code>        |

Frontend: Cloudflare Workers serving the compiled SPA. Backend: FastAPI on Railway. Postgres + Redis + Celery on Railway, separate per environment. Ollama (AI narrative generation) on a private Hetzner VM, reachable only over Tailscale — never exposed to the public internet.

### 6.4 Data residency (EU)

Production and staging both currently run on Railway infrastructure in **US East (Virginia, USA)** — confirmed directly from the Railway dashboard. **CostAdvisor does not today offer EU data residency.** If an EU-headquartered prospect or regulation requires it, this is a real infrastructure migration that has not been scoped as work yet, not a formality to confirm. See `jvpdocs/eu-data-residency.md` for the current state and the migration-plan template, and §11.4 below.

---

## 7. System Architecture — Technical Reference

---

## 7.1 The five things to understand before changing code

### 7.1.1 There are two costing paths, and one resolver they share

This trips up everyone. The repo prices things two different ways, with two different margin conventions.

**Path A — CostModel** (a team's own product, priced against their own formula). `CostModel` → `FormulaVersion` → `FormulaComponent`. Margin is a **separate field**: the engine strips it out to get a component base, computes indexed cost, then re-applies it ( `_component_base` / `_apply_margin` ).

**Path B — catalog combo** (a platform `FormulaTemplate` priced in one region). `FormulaTemplate` → `FormulaTemplateComponent` + `FormulaRegionCoverage` (one row per template × region × variant, carrying the base price and period). Margin is a **line inside the recipe**, so weights legitimately sum to 99.9–110 and `evaluate_weighted_template` rebases over the recipe's own weight sum. The level is exactly 100.0 at the base period by construction.

**Mixing the two conventions produces a number wrong by the margin.** `coverage.margin_pct` is descriptive — applying it on top would double-count.

The bridge between them is `services/formula_resolver.get_effective_lines(db, fv, cost_model)`, which returns `(EffectiveLine[], fallback_reason)`. A `FormulaVersion` may carry `source_coverage_id` + `link_mode`:

- `link_mode = NULL` (unlinked) or 'pinned' → build from the frozen `FormulaComponent` snapshot.
- `link_mode = 'tracking'` → resolve the linked catalog recipe **live**, at the cost model's current region.

**All six costing entry points route through it** — `_compute_indexed_cost`, `_compute_indexed_cost_detailed`, `calculate_evolution`, `calculate_brief`, `calculate_price_change`, `_compute_indexed_cost_forward`. That is why should-cost, breakdown, Evolution, Brief, Price-Change and the forward should-cost cannot disagree for the same formula version. **If you add a seventh consumer, route it through `get_effective_lines` too.**

It is also where **nested cost models** are expanded: a component of `component_type='model'` folds its child's lines in with weights multiplied. That is why nesting needed no change to `costing_engine.py` at all — every consumer inherited it. See §10.3 for what nesting means numerically.

### 7.1.2 The index resolution chain, in order

`data_resolver.get_single_index_value_detailed(commodity_id, region, year, quarter, team_id)` is where a number comes from. The order matters and each tier has a reason:

1. **composite** — an index computed live from other indices via an expression (cycle-guarded). A missing component yields `None`; it never fabricates a 0.
2. **fixed** — a team's `TeamIndexSource` with a constant value across all periods.
3. **team\_override / provider** — `IndexOverride`, exact region then GLOBAL. **A null override is a deliberate blank**, not a miss: it returns `None` rather than falling through to the scraped value.
4. **scraped\_region** — `IndexValue` at that exact region.
5. **scraped\_global** — the GLOBAL sentinel row.
6. **scraped\_any\_region** — any region with data for that commodity and period.
7. **monthly\_actual / monthly\_partial\_quarter** — the quarter mean of `IndexMonthlyValue` **actual** rows. Forecast rows are never used here; one reaching a historical should-cost would be fabrication. A quarter with fewer than three months is labelled partial rather than passed off as complete.

8. **scraped\_temporal\_carry\_forward** — the last observation before this period, carried forward. This tier exists so a future reference quarter does not flatten every ratio to 1.0.
9. **monthly\_carry\_forward** — same, from the monthly store.

Tier 7 matters more than it looks: 76 of the 98 commodities the catalog's cost lines reference are **monthly-only**, so before that tier existed three quarters of the catalog resolved to nothing and rode flat.

A component with no value **rides flat and produces an explicit data\_gaps entry**. It never silently becomes zero, and it is never silently omitted.

### 7.1.3 The index data model has three layers

The 2026-07 drop added a layer above the original one. Both are live.

- **TypeCode** — the label a cost line names ( `ELEC-EU` , `BRENT` ). Resolves to a series, or explicitly does not: `resolved` / `no_series` / `ambiguous` are three distinct states with three different remedies (run a scrape / buy a feed / decide what the code means). Never collapse them.
- **CommodityIndex** — the price series. Drop-loaded rows are identified by `commodity_key IS NOT NULL` . **Region is baked into the series key** ( `ammonia-eu` vs `ammonia-in` ), which is why they were not name-matched onto the older region-agnostic rows.
- **IndexCard** (display) + **IndexMonthlyValue** (the numbers). **Monthly is the source of truth; quarterly derives from it** — verified exactly derivable, 0.0000 max difference across all 1,516 quarterly rows.

Two traps: `-ppi` / `-wb` / `-mb` suffixes name a **source**, not a region — do not parse them. And `is_default_region` is deliberately **not unique**; 18 slugs ship several defaults, so the obvious constraint rejects the real data.

One number worth carrying in your head: **60 type codes resolve to brent** , carrying ~25% of all indexed cost weight. A breakdown that looks diversified can be one commodity reached through dozens of labels. That is what `/api/resolution` exists to show.

### 7.1.4 Tenancy is enforced at the database

Every tenant-facing table has a Postgres Row-Level Security policy ( `tenant_isolation` ). The active team is set per request in `app/database.py` . **Never write a query that bypasses it**. `bypass_rls_var` exists for Celery tasks, seed scripts and migrations only — and when you do set it, reset it in a `finally` .

Three tenancy shapes exist, and picking the wrong one is the most common serious bug here:

- **Strict tenant** — `team_id NOT NULL` , membership-scoped. Most things.
- **Platform-readable with team forks** — `team_id IS NULL` visible to all, team rows scoped. Used for `formula_templates` , `chemical_families` , `subfamilies` , editorial blocks, dimension terms, market signals. Under strict tenant the platform catalog would be invisible to everyone and look like a loader failure.
- **Platform-level, no RLS** — `commodity_indexes` , `producers` , calibration and projection runs. No `team_id` at all.

**Two real leaks were caught this way and both had the same cause:** a platform-scoped row has no team to CASCADE from, so it survives a test's tenant teardown and becomes live data in the next test's run. If you add a platform-scoped table, clean it up explicitly in tests.

### 7.1.5 Permissions: the plan ceiling runs *before* roles

`services/permissions.has_permission` evaluates in this order:

1. super-admin bypass
2. **plan ceiling** — a key absent from the team's plan is denied for everyone, whatever role they hold
3. custom team roles ( `Role` → `TeamMemberRole` )
4. membership-role fallback ( `owner` / `admin` / `member` ), **only if no custom role is assigned**

Consequences that have bitten this codebase already:

- **A new permission key must be granted to the Dream Plan** or the team owner is locked out of the feature you just shipped. The ceiling runs first.
- **A member with any custom role skips the fallback entirely**, so role grants are not optional extras.
- The fallback used to grant every `*.view` key by splitting on the last dot, which silently opened each new sensitive category as it was added. It now checks the category against `MEMBER_READABLE_CATEGORIES` — an **opt-in** list, so a new category is closed by default.

Platform-wide permissions are a separate axis: `has_platform_permission` + `UserPlatformRole` (Chemist, FX Manager, Content Editor, Support Agent). "May you write the library everybody reads" is a different question from "may you write your own copy".

## 7.2 Repo map

**Backend** — `backend/app/`

FastAPI at `main.py` . 44 routers, all prefixed `/api/` except auth ( `/auth/` ). 74 Alembic migrations, 63 test files.

The services worth knowing by name:

| File                                                          | What it owns                                                                                                                                                                |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>costing_engine.py</code>                                | should-cost, evolution, brief, squeeze, price-change, forward should-cost. The most complex file in the repo.                                                               |
| <code>formula_resolver.py</code>                              | <code>get_effective_lines</code> , <code>flatten_components</code> , <code>evaluate_weighted_template</code> , coverage fallback. The bridge between the two costing paths. |
| <code>data_resolver.py</code>                                 | the 9-tier resolution chain above, plus composites and forward values.                                                                                                      |
| <code>permissions.py</code>                                   | <code>has_permission</code> , <code>has_platform_permission</code> , the category opt-in list.                                                                              |
| <code>resolution.py</code> / <code>proxy_derivation.py</code> | type-code resolution, concentration queries, proxy execution, the swap backlog.                                                                                             |
| <code>intelligence.py</code>                                  | combo-grain derivation: level series, cycle position, seasonality blend, volatility percentile. Bounded query budget — 1 query regardless of window length.                 |
| <code>trigger_radar.py</code>                                 | negotiation windows from five feeds; contract notice deadlines.                                                                                                             |
| <code>trust.py</code> / <code>supplier_trust.py</code>        | the combo trust grade, and supplier grading resolved through the producer master.                                                                                           |

| File                                                                                                | What it owns                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>narrative.py</code> / <code>ollama.py</code>                                                  | LLM narrative. Best-effort, Tailscale-only, <code>llm_enabled=False</code> in production.                                                                                       |
| <code>incoterm_normalizer.py</code> / <code>fx_converter.py</code> / <code>unit_converter.py</code> | the comparability pipeline. Any pricing logic must account for all three.                                                                                                       |
| <code>drop/</code>                                                                                  | the 2026-07 data-drop reader, normalisation, authority rules, loaders.                                                                                                          |
| <code>sheet_roundtrip/</code>                                                                       | export → edit offline → reimport → diff → apply. Payload-agnostic; registered payloads for coverage pricing, dimension decisions and taxonomy reconciliation.                   |
| <code>price_list.py</code>                                                                          | matching a supplier price-list row to a cost model (exact/fuzzy/ambiguous/unmatched) and deriving its period. Parsing is <code>quote_extraction.py</code> 's, reused unchanged. |
| <code>negotiation_prep.py</code>                                                                    | checks a supplier's claim against the brief's own drivers. The verdict is never stored — it recomputes, because the driver's real movement changes as index data lands.         |
| <code>ai_cost_modeler.py</code>                                                                     | prompt, defensive parse, index resolution and the two promotion gates. Refuses rather than degrades.                                                                            |
| <code>index_region_coverage.py</code>                                                               | per-region sourcing facts off <code>IndexCard</code> . A silent series field is not a disagreement.                                                                             |
| <code>push.py</code>                                                                                | web push over VAPID. Best-effort like email; a 404/410 prunes the subscription.                                                                                                 |

**Data model spine.** `CostModel` → `FormulaVersion` → `FormulaComponent` is the team's formula. `ChemicalFamily` → `Subfamily` → `Product/FormulaTemplate` is the catalog taxonomy. `FormulaTemplate` → `FormulaTemplateComponent` + `FormulaRegionCoverage` is the catalog recipe. `AuditLog` records every mutation and is **append-only**.

## Frontend — `frontend/src/`

React 18 + React Router 6 SPA. Auth in `AuthContext.jsx` (Google OAuth + JWT in HttpOnly cookies). All HTTP goes through `api.js`. Charts are hand-written SVG components, not a chart library.

Navigation is an 8-tab journey shell: **Dashboard** → **Indexes** → **Portfolio** → **Monitor** → **Forecast** → **Negotiate** → **Intelligence** → **Team** (+ Admin for super-admins). Everything else lives in the account menu under "Go to".

Conventions, non-negotiable because the whole app follows them:

- `.ca-*` utility classes from `styles.css` plus inline styles. No CSS-in-JS, no Tailwind.
- **Every colour is a CSS variable.** Four themes switch by `data-theme`; a hardcoded hex breaks three of them. Several bugs of exactly this kind have been fixed — including `.ca-table th.right` being used 19 times and never defined, and `--danger` being referenced and never defined.
- New files go in `pages/` (full-page views), `components/` (reusable), `utils/` (pure helpers). No new top-level directories.
- `formatApiError(e)` for every error path. Raw API errors must not reach the UI.

- Read `DESIGN.md` first. The rules that catch people out: flat cards at rest (no shadow), no coloured border-stripes wider than 1px, mono for every number, the 7 commodity category colours are a **data vocabulary** and never UI chrome.

---

## 8. Operating the System (Local Development)

---

Full instructions in `jvpdocs/local-setup.md`. Prerequisites: Postgres and Redis running locally.

```
./start.sh                # backend :8000 + frontend :5173
cd backend && pytest        # full suite
cd backend && alembic upgrade head # migrations
cd frontend && npm run build  # must be clean before you commit
```

### 8.1 Test-environment gotchas that silently break RLS

1. **The DB role the app connects as must not be a superuser or have `BYPASSRLS`.** Postgres silently skips RLS for those roles, so every isolation test fails with "team B can see team A's data" while the policy is perfectly correct. If your container was created with `POSTGRES_USER=costadvisor`, that user is the cluster bootstrap superuser and **cannot demote itself**. Create a second ordinary role, grant it schema privileges, and point `DATABASE_URL` / `TEST_DATABASE_URL` at that.
2. **Two Fernet keys must be set** or the provider-credential and Google Calendar tests fail:  
`PROVIDER_CREDENTIAL_ENCRYPTION_KEY`, `GOOGLE_CALENDAR_ENCRYPTION_KEY`. Generate with `python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`.
3. **Python 3.14:** the pinned `pydantic-core` / `psycpg2-binary` / `sqlalchemy` versions have no wheels for it. Install newer versions of just those three locally — **do not change** `requirements.txt` without validating the real deploy target's Python version.
4. **The checked-in `backend/venv` is WSL-native.** On a Windows host it has `venv/bin/python` (an ELF binary) and no `venv/Scripts/python.exe`, so nothing runs from Git Bash or PowerShell. Drive it through WSL: `wsl.exe -e bash -lc 'cd /mnt/c/.../backend && ./venv/bin/python -m pytest -q'`. Postgres and Redis are inside WSL too, which is why `pg_isready` appears missing from the Windows side while the database is perfectly healthy.

Against a properly seeded database the suite is green (793 passed / 5 skipped at last full run; the skips are drop-dependent tests that skip cleanly without the data drop). Against a genuinely fresh database expect ~20 failures in `test_catalog_retargent` / `test_onboarding` / `test_seed_catalog` / `test_seed_combos` — those assume a pre-seeded DB state (the catalog workbook loaded, a super-admin seed user existing), not code bugs.

---

## 9. Operating Conventions & Known Pitfalls

---

Learned the hard way, each one from a real bug in this repo.

- **Never delete on silence.** A data drop is authoritative for what it covers and silent about the rest. Rows it does not mention are reported **stale**, never removed. Every loader is idempotent by comparison, never truncate-and-reload.

- **dry\_run** is the caller rolling back, not a flag threaded through the loader. One code path, so a dry run genuinely rehearses the real one.
- **Absent is not the same as empty.** A blank column in a source means "not stated" — write nothing. An early loader version treated it as "" and wiped two hand-set base prices.
- **Postgres treats every NULL as distinct in a unique constraint.** This has defeated upsert-in-place twice. If a nullable column is part of an upsert key, make it `NOT NULL DEFAULT ''` or fold it with `COALESCE` in a partial index.
- **A skipped check must never read as a clean one.** A validation check that finds nothing reports zero; it is not simply absent.
- **Never fabricate a number.** This is the product's whole value. A missing value rides flat with an explicit gap; a forecast is labelled a forecast with its method and vintage; a proxy is marked as a proxy. A fabricated "supplier's likely counter" was deleted for this reason (commit `03e0856` ), and a hardcoded  $\pm 1.5\%$  forecast band was removed twice.
- **Mutations are audited.** New write paths follow the `log_event` pattern. `audit_logs.team_id` is nullable — NULL means a platform-level event with no tenant, and the RLS policy keeps those invisible to tenants.
- **Alembic for every schema change.** Never modify tables by hand. Verify the downgrade too — every migration here has been up/down/up cycled.
- **A CHECK constraint can exist without being declared on the model class.** `ck_formula_components_component_type` was added by a migration, so reading `app/models/cost_model.py` says there is no constraint. If a new enum value inserts fine in your head and fails in Postgres, look for one the model never declared.
- **Correcting a migration after it has been applied does nothing.** Alembic has already recorded the revision, so the corrected body never runs — downgrade and re-upgrade. And remember the **test database is separate**: `DATABASE_URL="${APP_URL}_test" alembic downgrade -1 && DATABASE_URL="${APP_URL}_test" alembic upgrade head` . Both of these cost a debugging session each.
- **Pydantic schemas in `app/schemas/` are the API contract.** Keep them in sync with the ORM models.

## 10. Feature-Completeness & Mock-Data Audit

### 10.1 Audit summary (2026-10-01) — nothing is fabricated on the frontend

Six features shipped briefly (2026-09-27) as hardcoded-fixture `/preview/*` mockups behind an amber `PreviewBadge` , specifically so the specification (model, endpoints, behaviour assumed) was written down before the real build: nested cost models, the AI cost modeler, supplier price-list import, negotiation prep, per-region sourcing, and web push.

**Verified directly against the current code, not against commit messages:** `frontend/src/pages/preview/` no longer exists; `/preview` now routes to `WhatsLeft.jsx` (a real-gaps-only page); and every one of the six real surfaces calls its real backend endpoint —

| Feature            | Real surface                         | Real endpoint(s)                                |
|--------------------|--------------------------------------|-------------------------------------------------|
| Nested cost models | <code>CostModelBuilder.jsx</code>    | <code>GET /api/cost-models/{id}/nestable</code> |
| AI cost modeler    | <code>pages/AiCostModeler.jsx</code> | <code>/api/ai-cost-modeler/drafts</code>        |

| Feature             | Real surface                                         | Real endpoint(s)                                                     |
|---------------------|------------------------------------------------------|----------------------------------------------------------------------|
| Price-list import   | <code>pages/PriceListImport.jsx</code>               | <code>/api/price-lists/*</code>                                      |
| Negotiation prep    | <code>pages/workspace/NegotiationPrepArea.jsx</code> | <code>/api/negotiation-prep/*</code>                                 |
| Per-region sourcing | <code>pages/IndexSourcing.jsx</code>                 | <code>/api/indexes/region-coverage</code>                            |
| Web push            | <code>components/PushNotifications.jsx</code>        | <code>/api/push/*</code> + real <code>pushManager.subscribe()</code> |

A repo-wide grep for `mock / fixture / dummy / PreviewBadge / fabricat*` across `frontend/src` turns up only comments confirming the opposite (design-mockup references to `sample_idea/*.html` used purely for layout, and explicit notes that a previously-fabricated feature was deleted) — no live component renders invented data as if it were real. The one piece of genuine dead code found along the way, `_unused_PermissionForm` in `pages/Admin.jsx`, is unreferenced and never rendered — harmless, worth deleting whenever that file is next touched, not a data-integrity issue.

## 10.2 Honest, explicitly-labelled placeholders (not mocks)

Two surfaces remain flagged as not-yet-real — they say so plainly, which is the house style, not an oversight:

- The Forecast tab's composite headline-index chart has no 1:1 real-forecast substitute for its synthetic multi-index blend (see `CLAUDE.md`'s Scrum 21 entry); real per-series forecasts exist and are charted elsewhere ( `ForecastArea.jsx`'s own per-commodity cards, `/index-sourcing` ).
- Intelligence's "Product Intelligence" tab was flagged as a persistence-dependency placeholder when written; as of this snapshot it renders a real `ContextTab` fed by the editorial-block and dimension data that landed later (Units 7–8) rather than the original placeholder text — worth a direct look before assuming the old flag still applies if you touch that file.

## 10.3 Four rules the most recently-shipped features encode

Each one a newcomer would otherwise rediscover by breaking something.

1. **Nested cost models contribute composition, not price.** A `component_type='model'` line folds the child's lines into the parent with weights multiplied; the parent's `base_price` stays the anchor. Same convention as a chained `FormulaTemplate`, and what keeps weights summing to one. Cycles are refused at **save** ( `assert_valid_nesting` ) because a loop is unbounded recursion inside the engine. An unresolvable sub-model keeps its weight, rides flat and reports a data gap — dropping it would silently rescale everything else.
2. **A stored projection vintage goes stale against its own series.** Two of the five headline series are fitted to history that has since been overtaken, so a "projected" quarter can already be a fact. Anything consuming `/projections/latest` must drop projected points at or before the newest observation and say the vintage is behind — see `forward0f()` in `ForecastArea.jsx`.
3. **The AI cost modeler never writes a cost model, never invents an index, and never degrades to silence.** Promotion is gated on the recipe closing at 100% and every index line binding; an unresolvable suggested feed is flagged and blocks promotion by name; an unreachable model returns 503 rather than an empty draft that reads like a considered answer.
4. **Negotiation prep takes the supplier's position as an input.** The app holds no supplier-cost data and will not predict their counter. The should-cost has already consumed every verified index movement, so no verdict ever

presents a cited driver as grounds for paying more — the verdicts differ only in *how* a claim fails. The claim is stored; the verdict is recomputed each read.

---

## 11. Outstanding Items, Risks & Compliance Notes

---

### 11.1 Non-code items (an account, a signature, a dashboard, or a dataset — nothing a coding session closes)

| # | Item                                                                                                                                                                                          | What it blocks                                                  | Owner (§13)                                  |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------|
| 1 | Deploy sync — Railway repo was ~126 commits behind <code>dev</code> as of 2026-10-01 (not independently re-verifiable from this environment — see §6.1); nothing from Scrum 26 onward is live | Everything built since go-live                                  | Deploy/infra owner                           |
| 2 | SMTP credentials — no provider account chosen                                                                                                                                                 | Invites, alerts, demo confirmations fail silently               | Operations                                   |
| 3 | VAPID keys in production<br>( <code>VAPID_PUBLIC_KEY</code> / <code>VAPID_PRIVATE_KEY</code> ) — push works locally                                                                           | Push notifications                                              | Deploy/infra owner                           |
| 4 | Vendor DPA list + named incident-response contacts                                                                                                                                            | Security posture sign-off before any enterprise prospect review | Legal / compliance                           |
| 5 | Base-price anchors — 199 of 200 platform combos have no real price (import tooling is built)                                                                                                  | The catalog showing money, not just index levels                | Business / data owner                        |
| 6 | FD-1 feed mapping — remaining free-tier catalog commodities need a human to verify each live source                                                                                           | Most catalog commodities having any values at all               | Engineering (data verification, not a build) |
| 7 | Google Search Console verification + field Core Web Vitals                                                                                                                                    | SEO / landing-page validation                                   | Deploy/infra owner (blocked by #1)           |
| 8 | <code>SameSite=Strict</code> live cross-subdomain login verification                                                                                                                          | Confirming a Scrum 9 hardening assumption                       | Engineering (blocked by #1)                  |
| 9 | Branch protection on <code>main</code> — deliberately deferred                                                                                                                                | N/A — revisit once the team grows past 3                        | —                                            |

### 11.2 Small open engineering/analyst items (real, scoped, not done)

| #  | Item                                                                              | Note                                                                                                                                            |
|----|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 10 | Taxonomy reconciliation — 87 drop-vs-platform family/subfamily name mappings      | Mechanism built and tested ( <code>/api/sheets/taxonomy_reconciliation</code> ); nobody has filled in the decision sheet yet                    |
| 11 | Scrum 28 structured per-component min/max/yield fields                            | Currently written inline in advanced expressions (works); a dedicated UI is only worth it alongside a component-editor redesign                 |
| 12 | SEED-2 rebuild to the 2026-07 workbook                                            | Blocked on a matching combos data drop from whoever owns that workbook                                                                          |
| 13 | <code>seed_index_metadata.py</code> residue                                       | Marked SUPERSEDED but still runs against the pre-drop workbook                                                                                  |
| 14 | <code>proxy_derivation.derive_value</code> is idle                                | Live and correct; all 128 series with <code>proxy_logic</code> have a null operation/base_index — configuration gap, not code                   |
| 15 | <code>evaluate_all_alerts</code> Celery beat registration                         | Alerts fire on-demand only today, not nightly                                                                                                   |
| 16 | <code>ProductIntelligence</code> / <code>NarrativeReview</code> persistence model | AI narratives are Redis-cached only (7-day TTL); see §10.2                                                                                      |
| 17 | SCRUM-71 — buy-window lock/hold verdict at catalog-combo grain                    | Cost-model-grain version already shipped (Scrum 22); this is a distinct unbuilt variant — sequencing hazard: needs a combo-grain forecast first |
| 18 | CON-4 / CON-5 — editorial-content loader + staleness-recompute job                | Schema and read path (Unit 7) are built; the jobs that populate/refresh them are not                                                            |

### 11.3 Deliberately not doing, by design — do not re-open these as oversights

- Predicting a supplier's likely counter-offer. No supplier-cost data exists to do it honestly; a fabricated version was already built and removed once (commit `03e0856` ).
- Statistical cross-checking of two independent readings of one index series. We mostly hold only one reading per series; the few exceptions don't justify the machinery.
- Sheet-based BI/ERP export (SCRUM-69). Parking-Lot scope, never committed to this roadmap.

### 11.4 Compliance & data-protection note (EU)

- **Residency:** production and staging run in US East (Virginia). No EU hosting exists today. See §6.4 and `jvpdocs/eu-data-residency.md` .
- **Tenant isolation:** enforced at the database via Postgres RLS on every tenant table (§7.1.4) — independent of hosting region.
- **Vendor DPAs and incident-response contacts** are the two open items before `jvpdocs/security-posture.md` can be represented as complete to an enterprise or EU prospect (item 4, §11.1).
- This section is a pointer, not a substitute for legal/compliance review — read `jvpdocs/security-posture.md` , `jvpdocs/eu-data-residency.md` and `jvpdocs/vendor-risk.md` in full before making any representation to a prospect or regulator.

## 12. Architectural Follow-Ups

Not bugs, not blockers — places where the current shape is a deliberate, scoped choice that the *next* piece of work in that area should know about rather than rediscover:

- **The component editor has one more axis coming.** If item 11 (§11.2) is ever picked up, it changes `FormulaComponentItem` / `FormulaComponentOut` and the Reference Index column in `CostModelBuilder.jsx` a second time in the same release cycle as nested cost models did — plan both together rather than bolting one onto the other.
- **`EstimatorProposal` and the AI-cost-modeler's `AiCostDraft` are deliberately two tables, not one.** `EstimatorProposal` is keyed `(template_id, region)` and approves into the *catalog*; `AiCostDraft` approves into a team's own `FormulaVersion`. They share the draft-then-approve *rules*, never the table — don't merge them later for "simplicity"; that was tried in planning and rejected for good reason (two approve paths behind one row).
- **The editorial-block / dimension-term layer (Units 7–8) is additive infrastructure with no owner UI yet beyond Curation's review queues.** CON-4/CON-5 (§11.2, item 18) are the next real consumers; anything else that wants platform-authored prose or faceted tagging should read from this layer rather than inventing a second one.
- **Nested cost models and catalog template chaining are two separate recursion mechanisms with the same shape on purpose** (`formula_resolver.flatten_components` for templates, `get_effective_lines + component_type='model'` for cost models) — not yet unified, and probably shouldn't be: one is platform/team catalog data, the other is strictly tenant-owned, and RLS does not enforce the boundary between them at the FK level (§7.1.4) — a future unification would need to re-derive the same-team-only write-time guard currently living in `routers/cost_models.py`.
- **The Intelligence combo-grain engine ( `services/intelligence.py` ) and the costing engine ( `costing_engine.py` ) read index values through two different code paths** (`derive()`'s own bulk `IndexValue` / `IndexMonthlyValue` reads vs. `data_resolver`'s per-lookup resolution chain), reconciled only by a `value_sources.matches_costing_engine` disclosure flag rather than a shared code path — a deliberate query-budget tradeoff (one engine needs O(1) queries regardless of window length; the other needs the full 9-tier chain per lookup). Don't assume a bug if the two ever report different numbers for the same combo; check `value_sources.divergences` first.

## 13. Roles & Responsibilities

This table is intentionally left for the receiving team/organisation to complete before the handover is considered accepted — no names are assumed here.

| Role                          | Responsibility                                                                                                                | Assigned to |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------|
| Deploy / infrastructure owner | Railway + Cloudflare dashboard access; syncs the deploy repo (§6.1); sets production env vars (SMTP, VAPID — §11.1 items 1–3) | [TBD]       |
| Engineering lead              | Owns <code>CLAUDE.md</code> / <code>dev</code> branch; reviews and merges new work; owns the architectural follow-ups in §12  | [TBD]       |

| Role                  | Responsibility                                                                                                         | Assigned to |
|-----------------------|------------------------------------------------------------------------------------------------------------------------|-------------|
| Data / business owner | Sources real base prices for catalog combos; verifies FD-1 feed candidates live before they're wired (§11.1 items 5–6) | [TBD]       |
| Legal / compliance    | Vendor DPAs, incident-response contacts, any EU-residency commitment made to a prospect (§11.4)                        | [TBD]       |
| Operations            | Mail-provider account for SMTP (§11.1 item 2)                                                                          | [TBD]       |

---

## 14. Reference Documents / Appendices

---

- `CLAUDE.md` — the full TODO tracker, architecture conventions, security rules, and the readiness scorecard (every non-shipped item scored ease + value with reasoning). Read before starting anything.
- `jvpdocs/remaining-work-plan.md` — what is left (nothing buildable), plus a record of the three places an earlier plan's own assumptions turned out to be wrong about the data. Worth reading before trusting any ticket's description of a dataset.
- `jvpdocs/local-setup.md` , `development.md` — getting it running.
- `jvpdocs/wave1manual.md` — the complete non-code checklist for Wave 1.
- `jvpdocs/security-posture.md` , `eu-data-residency.md` , `backup-retention-policy.md` , `incident-response.md` , `vendor-risk.md` — security and compliance docs with explicit `[NEEDS CONFIRMATION]` / `[NEEDS ACTION]` placeholders.
- `jvpdocs/indexes-how-to-add.md` — adding a new index or scraper.
- `jvpdocs/custom_rbac.md` — the permission model in detail.
- `DESIGN.md` — the design system. Mandatory before UI work.
- `PRODUCT.md` , `technical-overview.md` — short external-facing summaries.
- `sample_idea/` — raw source material (ticket prompts, data drops, workbooks) most Wave 2/3 scrums were built against. Kept for provenance; large, not for browsing.

---

## 15. Sign-Off

---

This handover is considered complete once both parties confirm the content of this document (and its linked references) accurately reflects the state of the product and its outstanding items.

|                | Name | Role | Date | Signature |
|----------------|------|------|------|-----------|
| Handed over by |      |      |      |           |
| Accepted by    |      |      |      |           |